游戏王对战 AI：已核验项目与自建路线

核验日期：2026-10-18。方式：读取项目 GitHub 页面，并下载源码作静态检查；没有安装依赖、运行项目、训练模型或连接游戏客户端。以下“源码具备”不等于经本次运行验收；胜率与性能数字明确归于作者报告。

结论

确实存在开源的游戏王神经对战策略，但仅凭“有人开源”不能确认是哪一个。最值得核对原帖的两个候选是 sbl1996/ygo-agent 与 Noctfom/Galatea-Core。前者有可下载的训练权重，后者截至本次检查更新更近、中文工程文档更详细。二者都不是靠通用聊天模型逐句背卡文来完成所有裁定。

基本架构是：规则引擎与卡片 Lua 脚本执行决斗 → 解析当前玩家可见状态和合法选择 → 神经网络或规则程序选一个动作 → 引擎推进到下一次需要选择。一个完整 combo 是多次这样的决策，不是一次生成一段自然语言，再让游戏照做。

1. 最相关的四个项目

A. YGO Agent：最直接的已发布神经模型参考

- 仓库：https://github.com/sbl1996/ygo-agent
- 核验主分支提交：dbf5142d49aab2e6beb4150788d4fffec39ae3e5，2024-08-16，内容为预加载全部卡片/脚本及支持宣言卡名。
- 技术栈：C++ ygoenv，基于 envpool 与 ygopro-core；Python JAX/Flax；Transformer 特征编码及默认 LSTM；策略头、价值头；self-play 采样与 PPO clipping/GAE。训练脚本也留有 V-trace 等选项，不能把所有脚本一概叫同一种算法。
- 训练入口 scripts/cleanba.py；评估 scripts/eval.py；双模型比赛 scripts/battle.py；推理 ygoinf/ygoinf/server.py，JAX 与 TFLite 后端，README 展示 uvicorn 本地 API 与 Neos 集成。
- “LLM”用途：scripts/card/embedding.py 用 voyage-2 对卡名/卡文等生成 embedding，README 允许不用该特征。不是每个决策调用聊天大模型，也没有已证实的自然语言思维链展开器。
- 发布资产实查：0546_11300M.flax_model、0546_16500M.flax_model、0546_22750M.flax_model（各约16.6 MB）；0546_26550M.tflite（16.9 MB）；embed864.pkl；Python 3.10/3.11/3.12 环境二进制。https://github.com/sbl1996/ygo-agent/releases/tag/v0.1
- 牌组样例含青眼、烙印、白银城、炎王蛇眼、肃声、百夫长等。scripts/code_list.txt 当前有13,472行，但不能据此声称每张卡已测试，更不能把旧 embed864.pkl 或旧 checkpoint 当作完整覆盖全部新卡。README明确以 assets/deck 的已测试牌组为主。docs/support.md 还记载灵摆魔术师等限制；文档可能滞后，应逐卡做场景回归。
- 新卡障碍特别明确：Makefile 固定旧数据库提交和 scripts 提交8e7fde9；“现在网上有卡文”不会自动更新这里的可执行效果、特征词表或策略。
- 主项目 MIT；LICENSE 另包含 envpool 的 Apache-2.0 声明。底层引擎、脚本、数据库、卡图和商标还

需分别核授权，不能据此宣称整个发行包均可按MIT任意商业分发。

一手源码：

- https://github.com/sbl1996/ygo-agent/blob/main/scripts/cleanba.py
- https://github.com/sbl1996/ygo-agent/blob/main/ygoai/rl/jax/agent.py
- https://github.com/sbl1996/ygo-agent/blob/main/ygoenv/ygoenv/ygopro/ygopro.h
- https://github.com/sbl1996/ygo-agent/blob/main/docs/feature_engineering.md
- https://github.com/sbl1996/ygo-agent/blob/main/Makefile

B. Galatea-Core：较新的 PyTorch 工程框架，需严肃区分框架与棋力

- 仓库：https://github.com/Noctfom/Galatea-Core
- 核验主分支提交：0f7899e11a3436d86a86bc9b9107d6b36b48de1d，2026-10-12，v3.14.1 性能、解析及文档修复。GPL-3.0。
- 技术栈：公开 OCGCore DLL/SO；Python/PyTorch；CPU环境 workers + 共享观测/ZMQ + 中央 CPU/CUDA 批量推理；Transformer、Deck Encoder、FiLM 条件调制；PPO，带 GAE、KL 提前停止和辅助预测。
- trainer.py 确有联盟抽样逻辑：基础概率15%规则对手、25%历史模型（历史池存在时）、其余当前模型self-play；还设规则对手覆盖，因此这些概率不是保证最终每轮对局数恰好按比例。
- 关键入口 main.py、trainer.py、worker.py；环境 galatea_env.py；协议/状态 gamestate.py 与 core_trajectory.py；模型 galatea_net.py；竞技 arena_benchmark.py、model_versus.py；推理 ai_bot.py。
- 当前设计含精确追加式卡片词表、稳定 Main/Extra 构筑画像、动态局面、连锁上下文、最近发动、16步公开事件、公开提示；Side 不进入单局观测。卡片身份不是卡名字符串相似度。
- Lua 语义资产来自效果槽提取与代码编码。code_embedder.py 默认 SentenceTransformer all-MiniLM-L6-v2，生成384维代码向量。它是静态语义特征，并不执行规则；代码向量相似也不等于完整理解Lua行为。
- goal_planner.py 的“规划”是128维目标 latent，经未来资源/终局等辅助监督后回接策略和值；项目自己的分析明确说还不是搜索。不能因为叫planner、Transformer或者“逻辑推演引擎”，就说已经每步展开搜索树。
- 有一键包Release，但本次可核到的最新Release为v3.6.5，主分支已经v3.14.1；模型协议/资产版本不能混用。README称打包不包括本机模型。没有核实到可直接下载并具有独立强度验证的当前V4预训练checkpoint，因此应表述为“开源训练框架，是否附适配权重待确认”。
- 更新活跃并不代表已全面适配所有规则。公开观测文档明确仍不适配Type35整片墓地/卡组交换、Type161 Tag多人等边界，公开Core也未导出所有内部信息。

一手源码与文档：

- https://github.com/Noctfom/Galatea-Core/blob/main/trainer.py
- https://github.com/Noctfom/Galatea-Core/blob/main/goal_planner.py

- https://github.com/Noctfom/Galatea-Core/blob/main/code_embedder.py
- https://github.com/Noctfom/Galatea-Core/blob/main/docs/architecture.md
- https://github.com/Noctfom/Galatea-Core/blob/main/docs/public_observations.md
- https://github.com/Noctfom/Galatea-Core/releases

C. WindBot：最适合建立可解释的限定牌组基线

- 仓库：https://github.com/IceYGO/windbot
- 核验头提交2026-09-23（更新changelog），同日实质代码提交包括新增Elfnote牌组、更新枚举。MIT。
- C#程序接YGOPro/YGOSharp/SRVPro；Game/GameBehavior.cs解析服务器消息、维护可见局面；Game/GameAI.cs处理动作选择；Game/AI/Decks/每副牌一个或多个Executor。
- BlueEyesExecutor.cs可以直接看到按顺序注册 AddExecutor(Activate/Summon/SpSummon, CardId, 判断函数)；默认规则与专属条件决定展开、取对象、连锁响应。GameAI.OnSelectChain有实际入口。
- 这是人工编排策略，不是训练出来的模型；新牌组要补动作条件和优先级。给它任意.ydk不会自动获得同等棋力。
- 没有训练权重需求，CPU即可作为决策程序运行；不要凭此承诺任何牌组都强。
- https://github.com/IceYGO/windbot/blob/master/Game/GameAI.cs
- https://github.com/IceYGO/windbot/blob/master/Game/AI/Decks/BlueEyesExecutor.cs

D. M-YGO-Agent：新脚本适配分支，不能误当已成熟的新一代模型

- 仓库：https://github.com/KohakuBlueleaf/M-YGO-Agent
- 头提交2025-07-30为formatting，同日还有测试、词表和打包结构修改；MIT及继承的依赖声明。
- 继承ygo-agent，着重构建/目录修复，以及从MDPro3等YGOPro1/2衍生程序提取更新Lua脚本、生成code_list。示例含DragonMaidTail、TenpaiDragonMaid。
- README明确：EDOPro backend目前有问题，建议YGOPro-v1；录像不能被MDPro3正常解析；PyTorch RL实现仍在TODO。适合作为环境兼容参考，不能称为已完成的PyTorch替代方案或已发布强棋力新权重。

2. “会推combo、会应对对方”到底如何实现

合法动作层

OCGCore与Lua脚本负责实际效果、阶段、召唤条件、费用、对象范围、时点与连锁。环境读取MSG_SELECT_IDLECMD、MSG_SELECT_BATTLECMD、MSG_SELECT_CHAIN、MSG_SELECT_CARD等决策请求，将当前合法选项编码给策略。模型从候选中选，不应自由编造不存在的动作。

一次“发动某卡”后，可能还有支付代价、选卡、选择位置、选效果、确认等多个交互。多选可采用顺序选取或

受约束组合；同一套底层协议必须同时用于训练与实战推理。候选截断、padding mask、取消/结束选择的语义都必须回归测试，模型选合法索引并不自动保证适配层无错误。

展开与长期目标

规则bot：把已知展开线路写成条件优先级与状态标记，能快且可解释，遇到未编写分支容易弱。

RL策略：用局面、历史、构筑特征预测当前动作概率与局面价值，靠大量轨迹的结果逐渐学会多步配合。推理通常是一次网络前向选下一步，combo隐含在连续策略里；这不等于每步显式把所有后续路线模拟一遍。

搜索增强（自建建议，非上述项目已证实成熟功能）：在离线模拟器里对候选展开有限深度分支，以终场价值/胜率评分；可做beam search或MCTS。但要先解决可复制状态/重放、随机事件和隐藏信息，不可用模拟器知道的对手底牌作弊。YGO Agent虽有mcts目录，README仍将MCTS Planning列在Roadmap，不能据此宣称其发布权重推理使用成熟MCTS。

对方回合与隐藏信息

引擎在对方行动后的响应窗口同样发出合法选项，策略可选择手坑、速攻、陷阱、场上效果或pass；连锁结算由引擎完成。好策略还需学习“现在交干扰还是保留”，并非一见能发动就发动。

YGO Agent训练make_env明确oppo_info=False；环境对对手隐藏区域做遮罩，模型支持历史动作与LSTM记忆。Galatea维护公开事件、连锁与公开位置等有限历史。记忆公开过的信息不等于读取未知手牌。若要进一步做对手意图推断，需从公开行动估计可能牌组/手坑分布，对多个假设模拟；这是拟建能力，不能假定现有项目已经完成强对手建模。

LLM适合放在哪里

适合做卡文检索、教学解释、将人工combo整理成候选路线、辅助写测试；所有具体动作交由引擎验证。若做LLM规划工具，应给它结构化可见状态、合法动作ID和受限模拟接口，而不是直接让它根据卡名宣判效果。以上项目的embedding层不能被宣传成已经具备这套在线LLM规划能力。

3. 棋力、算力与复现：已知证据的正确读法

YGO Agent作者报告：GTX1650笔记本约1000环境steps/s，小青眼示例50次迭代已几乎全胜随机策略；但对其长期训练模型128局仅3.125%胜率。这说明随机基线非常弱，不是高手水平证据。长期模型作者称超过1亿局、相当于32张4090训练5天；另给多牌组典型8张4090与128核CPU数天的描述。这些是作者自报，不是本次复测，也不是自建项目的必需最低预算。

Galatea作者2026-10-10实验记录有训练分布内进步：后段对RuleBot逐轮胜率均值约85.49%，self-play约48.23%。文档自己明确这不是泛化证明或正式发布验收。它还报告超时、KL停止、吞吐等限制。不能拿训练中变动的对手和训练胜率，与另一个仓库的胜率直接比较。

建议独立评估：固定双方牌组和禁限表、固定代码/脚本/CDB/权重/词表哈希；先后手平衡；留出起手随机种子及未见对局；对规则bot、旧checkpoint、人工标注场景分别评估。记录胜负/平局/超时、非法响应/崩溃、整局决策耗时、关键干扰时点及具体combo成功率；报告样本数和不确定性。BO3/换备与BO1需分开。

源码、二进制、发布权重、配套语义资产都存在仍不等于一键可复现。YGO Agent依赖旧JAX上限和固定脚本；Galatea主分支与Release/模型协议有差距。先保存一套可运行版本，再更新卡池，远比同时追全部最新

版可控。

评估来源：

- https://github.com/sbl1996/ygo-agent/blob/main/README.md
- https://github.com/Noctfom/Galatea-Core/blob/main/docs/training_analysis_v4_initial.md
- https://github.com/Noctfom/Galatea-Core/blob/main/arena_benchmark.py

4. 推荐最小路线

1. 离线模拟器基线：先选一套OCG规则、日期明确的禁限表、两副固定牌组，锁定Core/CDB/Lua。验证完整对局、连锁、费用、多选、投降、重放及隐藏信息。用随机合法策略和WindBot/简单规则策略作为基线。
2. 限定卡组策略：先一副主牌组与少数常见对手，整理无阻抗展开和受一至两次常见干扰后的分支场景；全部由引擎回放验证。先实现稳定的策略观察与动作协议，再选择复用YGO Agent模型或Galatea框架。
3. 训练/搜索：可先用规则演示做模仿学习热启动（需自行实现/核对现成入口），再自博弈PPO；保持历史模型和规则对手，避免只会打自己。只有已有快照/可重放能力后再增加有限预算搜索。每次改动在固定评估集证明进步。
4. 扩卡池：对每张新卡完成CDB资料、可执行Lua、Core版本兼容、协议消息、词表与语义资产、场景测试、策略微调。扩到新构筑/新系列时做留出评估，不能只重建embedding就宣布会用。

若目标是“能解释展开并让你练应对”，先做离线固定牌组的对战+回放诊断，比从零训练全卡池通用AI更实际。若目标是当前OCG新卡竞技强度，维护脚本和测试集的工作量会持续存在。这里没有设计或执行Master Duel客户端自动操作；当前路线只需开源离线决斗引擎。

核验边界

尚未拿到用户所见项目链接，无法识别其实际指代；未发现并核实这些项目对应的独立同行评审棋力论文。仓库中的目标、界面截图、神经网络模块名都不等于能力证明。未执行项目，不保证所有README命令在当前依赖版本可运行。对当前新卡是否已支持，需要用户给具体牌表/卡名后逐卡核查。